

USD VS TW - 2024/2025

Arthur Pradier et Najib El khadir

Fiche descriptive

Nom: USD VS TW (UvsT): Ultimate Star Destroyer Versus The World

Genre: Shoot'em up, bullet hell

Description: Un shoot'em up exigeant inspiré des jeux d'arcades pionniers dans le genre. On y incarne un vaisseau spatial dont l'objectif est de détruire tous les vaisseaux spatiaux communistes du parti unique mondial de l'univers. Le but du jeu est de rester en vie le plus longtemps possible, en détruisant les ennemis qui apparaissent régulièrement, et en améliorant son vaisseau pour mieux affronter la difficulté grandissante.

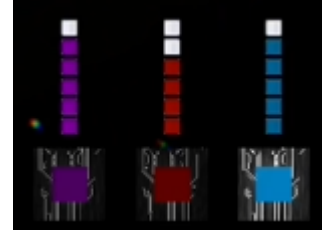
Inspirations: Raiden, Ikagura, Touhou



Game Design

L'objectif du jeu est de survivre aussi longtemps que possible à des vagues d'ennemis infinies. Pour cela, le joueur possède des armes, qui lui serviront à détruire les ennemis. Lors de leur mort, les ennemis peuvent lâcher des améliorations pour les armes du joueur. Lorsqu'un ennemi inflige des dégâts au joueur (avec des projectiles ou en touchant le joueur), le niveau de toutes les armes diminue d'un point.

Le joueur perd lorsqu'il prend des dégâts alors que ses armes n'ont plus qu'un seul niveau.



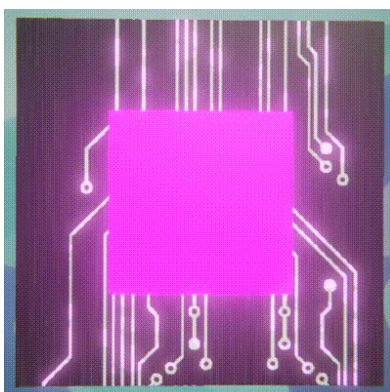
Armes

- Democracy Spreader: tire de nombreux projectiles dans un large cône devant le joueur.
- Liberty Beam: tire des projectiles en ligne droite devant le joueur, qui traversent les ennemis et infligent de lourds dégâts.
- Star Striper: des trous noirs orbitent régulièrement autour du joueur, créant des zones de dégâts qui détruisent ennemis et projectiles.



Améliorations des armes

Lorsque le joueur ramasse une amélioration d'arme, l'arme qu'il vient d'améliorer devient immédiatement son arme active.

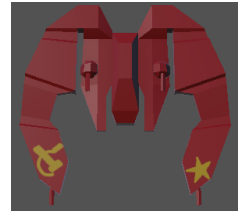


L'amélioration violette correspond à l'amélioration du Star Striper, le rouge au Democracy Spreader et le bleu au Liberty Beam.

Ennemis

Il existe 4 types différents d'ennemis, apparaissent par vagues autour de la zone de jeu :

- Le Zangrief est un gros vaisseau avec beaucoup de points de vie, qui tire un barrage de projectiles.



- Le Goblachev est un vaisseau qui décrit une trajectoire erratique dans la direction du joueur.

- La Kalashnicol est un petit vaisseau qui fonce sur le joueur en lui tirant dessus.



- Les Odnoklassniki sont des essaims de petits vaisseaux qui se déplacent en grands groupes et qui tirent des projectiles vers le joueur.

Les armes lâchent des améliorations à leur mort de manière aléatoire. Ils augmentent le score du joueur à leur mort.

Décor

- Des planètes défilent sur le fond de l'immensité spatiale
- Le joueur peut se déplacer dans la limite de l'espace de la caméra



Interface

l'interface est assez simpliste. Les **menus** comportent de la navigation basique entre les scènes. Pendant le jeu, appuyer sur echap ouvre le menu de pause, qui permet de **relancer la partie**, de **retourner au menu principal** ou de **gérer l'audio du jeu**.



L'interface **in game** nous donne **le niveau de chacune de nos armes** ainsi que **l'arme équipée actuellement**, le **numéro de la vague** d'ennemis courante, ainsi que le **temps survécu** et le **score**.



Assets utilisés et créés

- Mesh des vaisseaux
 - Téléchargés sur itch.io, Materials faits par Arthur, Prefabs faits par Najib.
- Projectiles
 - Shaders développés dans Unity à l'aide du Shader Graph par Arthur.
- Décors en arrière-plan (Espace, objets en fond avec un effet parallaxe)
 - Téléchargés sur <https://kenney.nl/>
- Audio
 - Musique originale, une variation par arme ainsi qu'une variation spéciale quand le joueur a toute sa vie ou peu de vie et une variation pour le menu pause. Réalisée et remixée par deux amis et Najib.

Code et architecture

Armes

Les armes se basent sur la classe abstraite `Weapon`, qui a un comportement de base et qui sera héritée par toutes les armes pour spécialiser certains comportements. Par défaut, `Weapon` appelle une coroutine qui appelle la méthode "Shoot" de manière régulière, en fonction de la vitesse de tir de l'arme. Pour gérer toutes les statistiques d'une arme, nous utilisons des `Scriptable Objects`, que nous utilisons comme container de données. Ainsi, chaque arme a une liste de `Scriptable Object` "Weapon Level" qui décrit ses statistiques par niveau. Lorsque l'on tire, ou que l'on génère un projectile, il nous suffit de lire les données du `Weapon Level` du niveau de l'arme.

Les implémentations d'armes sont relativement simplistes et servent surtout à définir le comportement de la méthode "Shoot" :

- `Linear` définit une arme qui tire tout droit
- `Radial` définir une arme qui tire en éventail
- `Circular` définir une arme dont les projectiles tournent autour du joueur.

En parlant de projectiles, les armes instanciant des projectiles quand elles tirent. Les projectiles ont chacun leur propre classe qui définit leur mouvement.

Comportements des ennemis

Les scripts contrôlant les ennemis héritent tous du script `BasicEnemy`, ils partagent certains fonctionnements communs, mais restent paramétrables et surchargent certaines fonctions leur permettant l'unicité de leurs modèles de déplacement et de tir, ainsi que le fonctionnement du système de drop d'améliorations.

Gestion des vagues

Le script `WaveManager` utilise des coroutines pour instancier une vague d'ennemis. Les caractéristiques de ces vagues (nombre d'ennemis par vague, temps entre les instanciations...) sont modifiés en fonction du numéro de la vague. L'augmentation du numéro de la vague entraîne également le scaling des caractéristiques des ennemis instanciés, en particulier toutes les 5 vagues.

Dégâts et interactions avec les ennemis

Les Prefab qui contiennent les projectiles ont tous un `collider`, de même que les joueurs et les ennemis. Quand un projectile ennemi ou un ennemi touche le joueur, il prend des dégâts, ce qui réduit de 1 le niveau de ses armes. Quand un projectile du joueur touche un projectile ennemi, le projectile ennemi est détruit.

Le projectile du joueur continue sa course jusqu'à rencontrer un ennemi ou quand il quitte le terrain. Il est alors détruit.

UI

Le singleton UIManager gère toutes les interactions avec l'UI. Il met à jour les données de l'interface quand les armes du joueur changent, quand les vagues d'ennemis avancent, etc.

Le script est aussi responsable d'afficher le menu pause. Pour stopper le jeu pendant la pause, on modifie la valeur de scale du deltaTime, que l'on met à 0. De cette manière, tous les mouvements dépendant d'un deltaTime s'arrêtent (soit tous les mouvements à l'écran).

Pour stopper tous les objets une fois la partie terminée, on utilise une Action, une fonctionnalité du C# qui permet d'utiliser un pattern Observer facilement. Tous les acteurs qui en ont besoin donnent à l'Action contenue dans UIManager une méthode de callback, que le manager dispatche lorsque la partie est terminée, ce qui lance la fonction dans les acteurs (dans notre cas, c'est généralement une fonction qui passe un flag isGameOver à true).

Audio

Le singleton AudioManager est responsable de jouer tous les sons du jeu. Il est conservé entre les scènes pour ne pas interrompre la musique. Le Gameobject qui contient l'AudioManager doit contenir 7 AudioSource: une pour jouer les SFX des armes, une pour les SFX généraux, et une pour chaque piste musicale.

Les SFX sont séparés en 2 AudioSource pour éviter de surcharger l'audio : comme les armes du joueur tirent énormément, l'AudioSource peut vite être saturé de bruit et devenir inaudible. Ainsi, les sons des armes qui sont lancés très régulièrement ont leur propre channel, et les SFX d'ambiance, plus rares, ont une autre AudioSource.

Pour gérer la musique, on appelle une méthode de l'AudioManager à chaque fois qu'une arme est mise à jour. Si l'arme courante a changé (ou si les niveaux de vie du joueur sont trop bas/haut), on réalise une interpolation linéaire du son de manière à transitionner d'une musique à une autre. Les musiques sont toutes lancées en même temps et on la même durée, donc transitionner d'une musique à une autre se fait bien et donne un effet très satisfaisant. Devoir utiliser 5 AudioSources reste tout de même peu pratique.

Répartition du travail

- Création / Récupération des assets (Arthur Pradier)
- Scrolling de la map (Arthur Pradier)
- Contrôleur du personnage (Arthur Pradier)
- Patterns et IA des ennemis (Najib El khadir)
- Fonctionnements des armes (Les 2)
- Game Manager (Les 2)
 - Spawn d'ennemis
 - Drop et récupération de powerups
 - Gestion du score
 - Timer et difficulté
- UI (Arthur Pradier)
- Musique et SFX (Les 2)
- Tunage des caractéristiques des ennemis, des vagues et des armes (Najib El khadir)

Améliorations possibles

- Combats de boss
- Plus de variations d'ennemis
- Aléas environnementaux (tempêtes solaires ? pluie d'astéroïdes ?)
- Plus de comportements pour les IAs des ennemis
- Plus d'armes
- Meilleure intégration audio (FMod, Wwise)
- Utilisation de pool d'instanciation pour les projectiles
- Meilleur scaling de la difficulté

Democracy will prevail...